Latest updates: https://dl.acm.org/doi/10.1145/3702652.3744220

RESEARCH-ARTICLE

Rubric Is All You Need: Improving LLM-Based Code Evaluation With Question-Specific Rubrics

ADITYA PATHAK, Birla Institute of Technology and Science, Pilani, Pilani, RJ, India
RACHIT GANDHI, Birla Institute of Technology and Science, Pilani, Pilani, RJ, India
VAIBHAV UTTAM, Birla Institute of Technology and Science, Pilani, Pilani, RJ, India
ARNAV RAMAMOORTHY, Birla Institute of Technology and Science, Pilani, Pilani, RJ, India
PRATYUSH GHOSH, Birla Institute of Technology and Science, Pilani, Pilani, RJ, India
AARYAN RAJ JINDAL, Birla Institute of Technology and Science, Pilani, Pilani, RJ, India
View all

Open Access Support provided by:
Birla Institute of Technology and Science, Pilani

PDF Download
3702652.3744220.pdf
08 April 2026
Total Citations: 4
Total Downloads: 4349

Published: 02 August 2025

Citation in BibTeX format

ICER 2025: ACM Conference on International Computing Education Research
August 3 - 6, 2025
Charlottesville, USA

Conference Sponsors:
SIGCSE

ICER '25: Proceedings of the 2025 ACM Conference on International Computing Education Research V.1 (August 2025)
https://doi.org/10.1145/3702652.3744220
ISBN: 9798400713408

Rubric Is All You Need: Improving LLM-Based Code Evaluation With Question-Specific Rubrics

Aditya Pathak*
BITS Pilani
Pilani, India
f20201448@pilani.bits-pilani.ac.in

Rachit Gandhi*
BITS Pilani
Pilani, India
f20201440@pilani.bits-pilani.ac.in

Vaibhav Uttam*
BITS Pilani
Pilani, India
f20200963@pilani.bits-pilani.ac.in

Arnav Ramamoorthy*
BITS Pilani
Pilani, India
f20220007@pilani.bits-pilani.ac.in

Pratyush Ghosh*
BITS Pilani
Pilani, India
f20221292@pilani.bits-pilani.ac.in

Aaryan Raj Jindal
BITS Pilani
Pilani, India
f20200781@pilani.bits-pilani.ac.in

Shreyash Verma
BITS Pilani
Pilani, India
f20211538@pilani.bits-pilani.ac.in

Aditya Mittal
BITS Pilani
Pilani, India
f20220569@pilani.bits-pilani.ac.in

Aashna Ased
BITS Pilani
Pilani, India
f20220536@pilani.bits-pilani.ac.in

Chirag Khatri
BITS Pilani
Pilani, India
f20221091@pilani.bits-pilani.ac.in

Yashwanth Nakka
BITS Pilani
Pilani, India
f20201082@pilani.bits-pilani.ac.in

Devansh
BITS Pilani
Pilani, India
f20202001@pilani.bits-pilani.ac.in

Jagat Sesh Challa
BITS Pilani
Pilani, India
jagatsesh@pilani.bits-pilani.ac.in

Dhruv Kumar
BITS Pilani
Pilani, India
dhruv.kumar@pilani.bits-pilani.ac.in

Abstract

Since the emergence of Large Language Models (LLMs) popularized by the release of GPT-3 and ChatGPT, LLMs have shown remarkable promise in programming-related tasks. While code generation using LLMs has become a popular field of research, code evaluation using LLMs remains under-explored. In this paper, we focus on LLM-based code evaluation and attempt to fill in the existing gaps. We propose multi-agentic novel approaches using *question-specific rubrics* tailored to the problem statement, arguing that these perform better for logical assessment than the existing approaches that use *question-agnostic rubrics*. To address the lack of suitable evaluation datasets, we introduce two datasets: a Data Structures and Algorithms dataset containing 150 student submissions from a popular Data Structures and Algorithms practice website, and an Object Oriented Programming dataset comprising 80 student submissions from undergraduate computer science courses. In addition to using standard metrics (Spearman Correlation, Cohen's Kappa), we additionally propose a new metric called as Leniency, which quantifies evaluation strictness relative to expert assessment. Our comprehensive analysis demonstrates that *question-specific rubrics* significantly enhance logical assessment of code in educational settings, providing better feedback aligned with instructional goals beyond mere syntactic correctness.

CCS Concepts

• **Social and professional topics → Student assessment**; • **Computing methodologies → Natural language processing**.

Keywords

Large Language Models, Code Assessment and Grading

ACM Reference Format:
Aditya Pathak, Rachit Gandhi, Vaibhav Uttam, Arnav Ramamoorthy, Pratyush Ghosh, Aaryan Raj Jindal, Shreyash Verma, Aditya Mittal, Aashna Ased, Chirag Khatri, Yashwanth Nakka, Devansh, Jagat Sesh Challa, and Dhruv Kumar. 2025. Rubric Is All You Need: Improving LLM-Based Code Evaluation With Question-Specific Rubrics. In *ACM Conference on International Computing Education Research V.1 (ICER 2025 Vol. 1), August 03–06, 2025, Charlottesville, VA, USA.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3702652.3744220

1 Introduction

The integration of Large Language Models (LLMs) into computing education has led to groundbreaking advancements, transforming both students and educators experiences [12, 14, 18, 37] . In computing education particularly, LLMs have demonstrated potential in generating instructional content [34, 39], providing personalized

*These authors contributed equally to this work.

This work is licensed under a Creative Commons Attribution 4.0 International License.

ICER 2025 Vol. 1, Charlottesville, VA, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1340-8/25/08
https://doi.org/10.1145/3702652.3744220

tutoring and doubt solving [22, 24] as well as assisting with code evaluation and grading [5, 17, 34]. Despite these advancements, there are significant gaps in using LLMs for code evaluation and grading. Most studies focus on generating feedback, helping students identify errors and improve their work [5, 32, 43]. Existing studies have demonstrated LLMs' ability to provide meaningful insights into code quality, syntax, and logic [6, 32]. Yet, grading, a critical and labor-intensive responsibility for instructors [27, 29], has received comparatively little attention [16, 21]. The evaluation and grading of student code is a critical component of computing education, as it helps instructors assess students' understanding of programming concepts and computational logic. This process consumes significant time and energy for instructors [27, 29], making it a prime candidate for automation.

Automated grading systems have been in use for a while [15, 28], but their effectiveness has often been limited due to their reliance on rigid test cases and simplistic evaluation criteria. Prior work in the domain of automated grading has focused on either introductory programming courses or on short answer evaluation [5, 37, 45]. Moreover, we note that existing work on feedback and grading of programming assignments has focused on *question-agnostic (QA) rubrics* [10, 34], emphasizing generic criteria such as correctness and syntax across diverse problems, but in reality we find that the human instructors actually use *question-specific (QS) rubrics*.

In this paper, we focus on evaluating the effectiveness of LLMs in grading student code using *question-specific rubrics* in more advanced computing courses, such as Object-Oriented Programming (OOP) and Data Structures and Algorithms (DSA), where complex problem-solving demands greater precision and context-awareness. We hypothesize that such an approach utilizing **question-specific rubrics** would yield more accurate evaluations compared to question-agnostic methods. Thus, the primary research question we seek to answer in this paper is:

"How effectively can an LLM-based grader evaluate student code using a question-specific rubric compared to a question-agnostic rubric? What measurable differences exist in evaluation quality and feedback specificity between these two approaches?"

We propose three novel techniques for code evaluation: (1) **Complete Rubric Evaluation (CRE)**, is a LLM-agent which assesses student submissions against the entire rubric, prioritizing logical correctness and intentionally overlooking syntax errors, to focus primarily on conceptual understanding. A deterministic compiler-equipped agent is used for checking syntactical correctness ; (2) **Pointwise Rubric Evaluation (PRE)**, is similar to CRE, but evaluates submissions by individually checking each criterion within the rubric, providing detailed and granular feedback. It is comparatively more resource-intensive; and (3) **Ensembling Method Evaluation (EME)**, which enhances reliability by aggregating rubric-based evaluations through majority voting and related ensemble mechanisms, ensuring robust feedback and assessment consistency. Additionally, we introduce a new evaluation metric called **Leniency**, which measures the strictness or leniency of automated evaluations compared to expert human assessments, providing insights into the relative evaluation rigor.

Our empirical findings demonstrate that question-specific rubrics substantially outperform question-agnostic rubrics, leading to improved accuracy, feedback relevance, and alignment with educational objectives.

Conducting this research required a suitable dataset containing student code submissions, model solutions, grading rubrics and feedback. We found that no such dataset was available publicly. To fill this gap, we created our own dataset, drawing from student submissions for OOP and DSA programming exercises. This dataset includes: (1) problem description, (2) student-submitted code, (3) model solutions, (4) grading rubrics, and (5) qualitative feedback, all of which are essential for benchmarking the performance of LLMs in code evaluation tasks. We will be releasing this dataset soon to the public to facilitate further research in this area. The main research contributions of this work are as follows:

- We present a new dataset that comprises submissions from two important courses in undergraduate computing education: (1) Object-Oriented Programming (OOP), and (2) Data Structures and Algorithms (DSA). (**§3**)
- We introduce three novel techniques for code evaluation and grading: (1) Complete Rubric Evaluation (CRE), (2) Pointwise Rubric Evaluation (PRE) and (3) Ensembling Method Evaluation (EME). (**§4**)
- We present a new metric that measures the strictness or leniency of an evaluation system based on rubric. (**§5**)
- We perform a comprehensive evaluation of the proposed techniques showing that our proposed techniques outperform all other techniques on both the datasets, achieving high correlation with human graders. (**§6**)

By demonstrating the effectiveness of question-specific rubrics in LLM-based grading systems, we pave the way for more accurate and efficient automated evaluation tools. These tools have the potential to save instructors significant time and effort, allowing them to focus on other aspects of teaching and mentoring. Additionally, students stand to benefit from more detailed and contextually relevant feedback, which can help them identify areas for improvement and deepen their understanding of programming concepts. We have made our dataset and code publicly available on HuggingFace[1] and GitHub[2] respectively.

2 Related Work

Effective assessment and feedback mechanisms are fundamental to programming education, serving as critical scaffolds that allow for guiding students through the problem-solving process with proper guidance. Prather et al. [36] emphasize that effective feedback extending beyond the binary correct/incorrect judgments based on test-cases, generally provided by traditional automated grading systems is crucial for fostering self-regulated learning. Similarly, Hao and Tsikerdekis [11] note that well-designed feedback promotes metacognitive development. The increasing enrollment in programming courses has rendered grading the programming assignments increasingly labor-intensive and time consuming [25]. To address this challenge, instructors frequently rely on automated grading tools [20].

[1]https://huggingface.co/datasets/BITS-Pilani-GRC/RubricEval
[2]https://github.com/BITS-Pilani-GRC/Rubric-Grader

Technique	Grading	Feedback	Programming Focused	Question Agnostic Rubric	Question Specific Rubric
BLEU [33]	✓	×	×	×	×
CodeBLEU [38]	✓	×	✓	×	×
CodeBERTScore [46]	✓	×	✓	×	×
ICE-Score [47]	✓	×	✓	×	×
CodeJudge [41]	✓	✓	✓	×	×
Phung et al[34]	✓	✓	✓	✓	×
CodEv [42]	✓	✓	✓	✓	×
Proposed Techniques	✓	✓	✓	✓	✓

Table 1: Comparison of LLM-Based Techniques for Grading and Feedback

2.1 Limitations of Traditional Auto-grading Approaches

Conventional autograding systems have primarily relied on predefined test cases to evaluate student submissions. Lobb et al. [26] developed a system that executed student code and compared outputs against expected results derived from instructor-defined test cases. While this approach allowed for basic assessment of functional correctness, these systems often lack the ability to understand underlying semantic errors or provide nuanced explanations [19].

Additional limitation of test-case-based approaches is the burden placed on instructors to create comprehensive test suites manually. Keuning et al [19] highlighted the challenges associated with generating test cases that effectively cover edge cases and potential error conditions. Furthermore, traditional systems often focus exclusively on functional correctness, neglecting aspects such as code style, efficiency, and adherence to best practices [35]. Finally, test suites require successful compilation where even minor syntactical errors can render logically correct code non-functional. This results in an incomplete assessment of student programming abilities and fails to promote holistic coding skills development.

Our approach addresses these challenges by employing instructor-defined rubrics that provide specific evaluation criteria. This enables more meaningful feedback beyond binary judgments while maintaining scalability. Though rubrics are also manually intensive, they have great upside in helping instructors evaluate students consistently and objectively [8], while providing students with clear expectations and constructive feedback to identify their strengths and weaknesses [31].

2.2 Emergence of LLMs in Programming Assessment

The emergence of Large Language Models (LLMs) marks a significant paradigm shift in autograding approaches. Recent studies by Denny et al. [23] have outlined how LLMs can leverage their extensive training data to grasp nuances of code, identify common errors, and explain complex concepts in an accessible manner. Multiple studies [7, 21, 34] have shown that advanced LLMs such as GPT-4 [30] can approach human-level feedback quality and deliver formative insights beyond simple correctness checks. Furthermore, Leinonen et al. [13] analyzed techniques for using LLMs to improve programming error messages, making them more informative and actionable for students.

Though the above work highlights the potential of LLMs in the field, many studies also raised questions on the reliability and consistency of LLM-based evaluation [40]. While the majority of the existing work focuses purely on providing effective feedback, we comprehensively also cover grading the assignments which is essential for both instructors and students, especially for a large-scale programming course. Our approach using question-specific rubrics also ensures structured evaluation criteria that combine human expertise with AI capabilities to provide consistent, contextual feedback.

2.3 LLM-based Techniques for Grading and Feedback

Table 1 compares the major LLM-based approaches for programming assessment. CodeBERTScore [46] leverages pre-trained BERT [9] models to encode semantic vectors of reference and generated code, measuring the similarity between these vectors rather than relying on token-matching approaches. This method improved upon earlier techniques like traditional BLEU [33] (which treated code as mere text) and its specialized derivative CodeBLEU [38] (which incorporated weighted n-gram matching, AST comparison, and data-flow analysis). Despite CodeBERTScore's innovations in capturing semantic meaning, it still faces a fundamental limitation: context similarity doesn't necessarily represent semantic similarity, resulting in suboptimal performance when evaluating functionally identical code implemented with different approaches which is a particular challenge in educational environments.

ICE-Score [47] has explored using LLMs directly for code evaluation without relying on test cases. However, this approach demonstrates limited correlation with human judgment and remains susceptible to LLM hallucinations, particularly when evaluating complex code with intricate semantics. Similarly, CodeJudge [41] leverages "slow thinking" to guide LLMs in evaluating code semantics. By decomposing evaluation into step-by-step analysis followed by summarization, and introducing a taxonomy of code inconsistencies with severity levels, it achieves better correlation with semantic correctness compared to other methods. The above mentioned techniques are primarily for benchmarking the correctness and quality of code generated by LLMs but are unable to generate any meaningful feedback for student code submissions.

Beyond these approaches, we examined rubric-based techniques for code evaluation and identified a significant gap in research exploring such methods. Although rubric creation is as time-intensive for instructors as building test suites, the resulting rubrics provide far greater educational value to students. Unlike test suites, which provide only error-based feedback and do not work when the student code does not compile, rubrics deliver direct, structured feedback that systematically supports both learning and assessment. For ease of our discussion, we further subdivide rubric-based programming evaluation on the basis of rubric specificity: **(1) question-agnostic rubric** and **(2) question-specific rubric**. These are formally defined in Figure 1.

LLM-based techniques using question-agnostic rubrics often fail to capture the nuances of specific programming problems, leading to misaligned evaluations, and still struggle with grading feedback compared to human tutors [34]. Phung et al. [34] showed that both GPT-4 and ChatGPT frequently misidentify code issues and incorrectly assign points for general correctness and edge cases when using general rubrics, suggesting question-specific rubrics might be more effective for automated assessment systems. More recent developments include domain-specific approaches like CodEV [42], which leverage LLMs, Chain of Thought, LLM ensembles, and a question-agnostic rubric to improve score accuracy and consistency.

Xie et al. [44] propose a multi-agent "Grade Like a Human" system which creates context-aware rubrics for short-answer questions. It then uses these rubrics to score responses, give tailored feedback, and run accuracy checks. Because it targets short-answer grading, this approach does not transfer directly to programming assessments.

To the best of our knowledge, this study is the first to utilize question-specific rubrics for grading and providing feedback on student code submissions. Our method distinctly separates the assessment of logical reasoning from syntactic correctness, mirroring real-world practice where conceptual understanding is given preference over code syntax.

3 Dataset

We present two datasets, one based on Object Oriented Programming (OOP) and second based on Data Structures and Algorithms (DSA). These courses are taken by all Computer Science undergraduates making them relevant to our study. Overall, our dataset contains 230 student submissions (80 from OOP and 150 from DSA). Full Dataset is available on HuggingFace [3]. Below, we provide the high level details about the dataset:

3.1 OOP Dataset Construction

For the OOP dataset, we rely on student submissions as part of a programming exam conducted in an OOP course at BITS Pilani in Fall 2024. The programming exam consisted of one Java programming question which further consisted of seven methods which needed to be implemented by the students. Students were provided with a structured starter template that included class declarations, helper functions, and predefined method scopes for implementation. A main function was also supplied to the students for testing and validating their solutions. The tasks in these methods focussed on applying object-oriented programming principles to handle file I/O, data filtering, and data updates.

We selected 80 student submissions from a total of 350 submissions, all from undergraduate sophomores. The submissions were graded by Teaching Assistants (TAs) and categorized into four score ranges: 0–10, 10–20, 20–30, and 30–35 marks (max score was 35). From each category, 20 solutions were selected at random, totaling to 80 student submissions. The dataset consists of the following components:

(1) **Detailed Problem Statement:** A detailed problem statement provided by the instructor, outlining clear, step-by-step instructions for implementing each function in every part of the question.
(2) **Scaffold Code:** The code provides a foundational class structure with templates and clear markers indicating where students should insert their code.
(3) **Rubric:** The rubric outlines each step as a distinct component, assigned either 1 or 2 marks, with no partial grading. A 2-mark step is awarded either 0 or 2 marks. Each of the seven subproblems follows a structured sequence, with steps arranged in a logical order based on their implementation in the codebase.
(4) **Model Solution:** A model solution prepared by the instructor, which includes implementations of all steps while strictly adhering to the guidelines in the problem statement and rubric.
(5) **Student Submissions:** Each student submission consists of three parts:
   - **Student code:** The student's submitted Java code.
   - **Grades:** Part-by-part evaluation conducted through a consensus-driven approach by two graders.
   - **Feedback:** For each of the seven subproblems, a comprehensive feedback was created collaboratively by both graders based on the problem statement and rubric. Both graders are final-year students with extensive experience in programming courses, ensuring a thorough and consistent assessment process.

For example, in the student code in Figure 2, in line 2, List<Player> players = new List<Player>(); is incorrect, as List is an interface and cannot be instantiated directly (Step 1). However, all subsequent steps are correct. The final score, determined by human annotators, is 8 out of 9, with individual step-wise marks assigned as 0, 1, 1, 1, 1, 1, 1, 1, 1.

3.2 DSA Dataset Construction

To ensure diversity and comprehensiveness, we selected DSA problems from the Geeks for Geeks (GFG) practice website [7], spanning 9 topics and 3 difficulty levels (easy, medium, and hard). The distribution of problems across topics and difficulty levels is shown in Figure 3 and Figure 4. For each problem, the dataset contains:

(1) **Problem Statement** - The problem statements were sourced from the descriptions provided on the website for each question. These include textual description of the problem, input size constraints and example input-output pairs to illustrate expected behavior.
(2) **Model Solution** - One model solution was taken from the editorial given with the problem.

[3] https://huggingface.co/datasets/BITS-Pilani-GRC/RubricEval

Rubric Types for Programming Problems: For a programming problem P with description D and expected solutions S), we define:

- A **question-agnostic rubric** R_{agnostic} as a set of evaluation criteria independent of $specific - logical\ requirements$. Formally, $R_{\text{agnostic}}(P_1) = R_{\text{agnostic}}(P_2)$ for any distinct problems P_1, P_2. These criteria often cover general aspects like code style, basic correctness, and efficiency.
- A **question-specific rubric** $R_{\text{specific}}(P)$ as a set of evaluation criteria tailored to $unique\ logical - requirements$ and $constraints$. Formally, $R_{\text{specific}}(P_1) \neq R_{\text{specific}}(P_2)$ for distinct problems P_1, P_2, with criteria directly referencing D and S.

Figure 1: Formal definitions of rubric types used in LLM-based code evaluation.

```
CricketDataHandler: readPlayersFromFile Method [9 marks]
Write code for reading player data from the input CSV
file and creating
a list of Player objects.
• Step 1: Create an empty list to store player details.[1 mark]
• Step 2: Open the specified file for reading data. [1 mark]
• Step 3: Ignore the first line since it contains the
  column names. [1 mark]
• Step 4: Read each line one by one until reaching the end of the
  file. [1 mark]
• Step 5: Split the line into different pieces of information.
  [1 mark]
• Step 6: Create a new player using this information.[1 mark]
• Step 7: Add the new player to the list. [1 mark]
• Step 8: Close the file after reading all data. [1 mark]
• Step 9: Return the complete list of players. [1 mark]
```

```java
public List<player> readPlayersFromFile(String fileName
    ) throwsIOException {
List<player> players=new List<player>();
Scanner sc= null;
Sc = new Scanner(new FileInputStream("InputCricketData.
    csv"));
sc.nextLine();
while(sc.hasNext()){
    Player p = new Player();
    String a=sc.nextLine();
    String s1=a.split(",")[0];
    String s2=a.split(",")[1];
    String s3=a.split(",")[2];
    String s4=a.split(",")[3];
    String s5=a.split(",")[4];
    p.setPlayerName(s1);
    p.setRole(s2);
    p.setRunsScored(Integer.parseInt(s3));
    p.setWickets Taken (Integer.parseInt(s4));
    p.setTeamName(s5);
    players.add(p);}
    sc.close();
    return players;}
```

Figure 2: Sample Question from OOP dataset and corresponding student submission

(3) **Submissions** - Six distinct submissions were selected for each problem, representing different categories of outcomes : Correct (3 solutions), Wrong (1 solution), TLE (1 solution) and Compilation error (1 solution).

(4) **Rubric** - The rubric was designed by considering different approaches to solving the question. Marks were assigned to each step based on its relevance and significance across various possible solutions.

(5) **Feedback and Marking** - Two human graders collaboratively evaluated six different solutions per problem, following the predefined rubric. They also provided detailed feedback for each implementation step. Both graders, as final-year students with extensive programming experience, ensured a thorough and consistent assessment process.

Topic	Number of questions
Arrays	4
Binary Search	3
Bit Magic (Bitwise Operators)	2
Dynamic Programming	2
Graphs	3
Hash	3
Linked Lists	2
Strings	4
Trees	2

Figure 3: Distribution of Problems by Topic

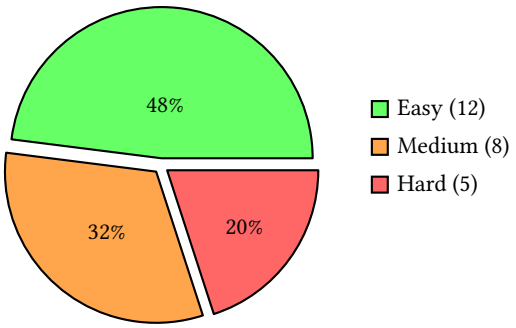

Figure 4: Distribution of Problems by Difficulty Level

A sample comprising student solution, problem statement, rubric, and grade & feedback is shared in Figure 5, Figure 6, Figure 7, and Figure 8 respectively.

4 Proposed Techniques

4.1 Motivation

The proposed techniques discussed below were designed in order to evaluate submissions using question-specific rubrics. As discussed

```java
class Solution {
    // Function is to check whether two strings are anagram of each other or not.
    public static boolean areAnagrams (String s1, String s2) {
        // Your code here
        if(s1.length() != s2.length() )
            return false;
        int charcount[] = new int[256];
        for(int i = 0; i < s1.length(); i++ ){
            charCount[s1.charAt(i)]++;
            charCount[s2.charAt(i)]--;
        }
        for(int count: charCount)
            if( count != 0)
                return false;
        return true;
    }
}
```

Figure 5: Sample Student Solution for DSA

earlier, question-specific rubrics were introduced in order to emulate college or university-level grading of student code. Akin to college grading, we break down the problem statement into steps that the student is expected to implement. Marks are awarded with respect to the student's approach for each specific step. The marking is binary, so a student is awarded either full or zero marks for a step. Our datasets attempt to emulate a similar level of leniency and specificity with grading of the ground truth values.

4.2 Complete Rubric Evaluation (CRE)

Complete Rubric Evaluation (CRE) is a system of rubric-based evaluation developed as part of our research into LLMs and their capabilities in code evaluation. As shown in **Figure 9**, the CRE LLM grader agent takes as input the complete problem description followed by a complete rubric (a multi-tiered point-by-point marking scheme). The rubric points act as anchors around which the student code is evaluated. Finally, the grader agent takes in the entire student code file, including all methods and classes implemented by the student.

The grader returns a single JSON dictionary without additional text. The JSON dictionary is a nested rubric, where the primary keys are method names. The values may either be the marks assigned to the method or another dictionary containing finer evaluation points for the method. Since LLMs are unreliable in arithmetic operations, the final calculation of marks is performed by a recursive function outside the LLM loop.

The LLM in CRE grader is instructed to ignore syntax errors in the student code, treating them as correct. CRE aims to infer the logical intention behind student code and mark it accordingly. This simulates a university-like evaluation environment, where logical reasoning is prioritized over minor syntax errors. This approach addresses gaps in traditional evaluation methods (e.g., test-case-based evaluation), which assign zero marks to submissions with minor errors.

While the LLM performs only logical evaluation, syntax evaluation is conducted separately using a deterministic compiler-equipped agent. The student code is executed by the agent using a compiler via a system call in a Python script. The compiler returns a syntax assessment, and the agent uses a penalty-based system to assign syntax marks accordingly. For instance, with a maximum of 5 syntax marks and a penalty of 0.5, a code snippet with 5 syntax errors is awarded 2.5 marks. The final student marks are the sum of logical and syntactical scores. Prompts used for instructing LLMs are shared in Appendix A.

4.3 Pointwise Rubric Evaluation (PRE)

Pointwise Rubric Evaluation (PRE) is a modified version of CRE. Instead of evaluating the entire rubric at once, the PRE LLM grader agent assesses the student code based on a single rubric point at a time. PRE Grader takes as input the problem statement, student code, and a single rubric point for evaluation. It returns a JSON dictionary containing the rubric point as the key and the assigned LLM marks as the value. These results are stored in a JSON file for later computations. PRE is resource-intensive due to multiple API calls per student solution. An LLM call is executed in order to evaluate each point in the rubric, consuming tokens and time.

4.4 Ensembling Method Evaluation (EME)

Inspired by CodEv [42], Ensembling Method Evaluation (EME) leverages large language models (LLMs), such as GPT-4o, Claude 3.7 Sonnet [1] and GPT-4o mini, to validate ensemble-generated results using a structured evaluation framework. The method employs a sampling and voting-based approach, primarily relying on the majority voting method to determine the final ensemble output. In cases where no clear majority emerges, the rounded mean method is used as an alternative to aggregate the scores effectively. Additionally, EME incorporates a feedback selection mechanism, where the system identifies the most representative feedback by selecting the evaluation feedback whose total score is closest to the final ensemble score. As shown in **Figure 10**, this technique takes the question, the student solution, the reference solution and the rubric. In case of DSA dataset we also add a approach identification prompt which uses the GPT 4o model, to identify the which approach the student has used from the rubric. This prompt provides us with a confidence interval for the approach identified by the model.

5 Metrics and Evaluation

5.1 LLM selection

In order to maintain a balance between LLM accuracy and token costs, we used OpenAI's GPT-4o-mini model to conduct our evaluations. We also used Anthropic AI's Claude 3.7 Sonnet model with Extended Thinking deactivated.

5.2 Data preprocessing

5.2.1 Scaling. With varying scales and degrees of rubrics, the scale at which grades are awarded is different for every technique. Here, scale refers to the maximum marks or score that can be awarded for the evaluation technique . In the OOP dataset, the question-specific rubric awards up to 35 points, whereas the question-agnostic rubric compresses the same outcome space into a holistic 5-point scale. Moreover, when evaluated method-by-method, a question-agnostic rubric evaluates each method on the same 0-4 scale, whereas in the original rubric, each method holds different weights in terms of

```
Given two strings s1 and s2 consisting of lowercase characters. The task is to check whether two given
strings are an anagram of each other or not. An anagram of a string is another string that contains the same
characters, only the order of characters can be different. For example, "act" and "tac" are an anagram of
each other. Strings s1 and s2 can only contain lowercase alphabets.
Note: You can assume both the strings s1 & s2 are non-empty.
Examples:
Input: s1 = "geeks", s2 = "kseeg"
Output: true
Explanation: Both the string have same characters with same frequency. So, they are anagrams.
Input: s1 = "allergy", s2 = "allergic"
Output: false
Explanation: Characters in both the strings are not same, so they are not anagrams.
Input: s1 = "g", s2 = "g"
Output: true
Explanation: Character in both the strings are same, so they are anagrams.
Constraints: 1≤ s1.size(), s2.size() ≤ 105
```

Figure 6: Sample Problem Statement for DSA

```
Anagram:
1. Check if two Strings are Anagrams of each other
2. Solution 1:
  1. Initialize a map or dictionary or array to store the character frequencies. Initially, the frequency
     for each character should be zero.[1 mark]
  2. For each character in the first string, the corresponding frequency is incremented by 1. [1 mark]
  3. For each character in the second string, decrement its corresponding frequency by 1. [1 mark]
  4. Iterate through the entire map or dictionary or array. If any frequency is non-zero, then return false. Else return
    true. [1 mark]
3. Solution 2:
  1. Initialize a map or dictionary or array to store the character frequencies. Initially, the frequency
     for each character should be zero. Keep separate data structures for each of the strings. [1 mark]
  2. For each character in the first string, the corresponding frequency in its data structure is
     incremented by 1. [1 mark]
  3. For each character in the second string, the corresponding frequency in its data structure is
     incremented by 1. [1 mark]
  4. Iterate through both the data structures. If the frequency for any unique character in both the
     data structures does not match, return false. Else return true. [1 mark]
4. Solution 3:
  1. Sort both the strings. [2 marks]
  2. Return true if both the sorted strings are exactly same. Else, return false. [2 marks]
```

Figure 7: Sample Rubric for DSA

maximum marks that can be awarded for that method. Therefore, in order to make quantitative comparisons between the two datasets, we scale the grades awarded by evaluation on the question-agnostic rubric to match the 35 mark question-specific rubric design. For the purposes of understanding, we shall refer to the Ground Truth values as the Base Dataset (B) and the LLM evaluations as the Experiment Dataset (E). B and E contain N data points, one for each student being included in the assessment. $B[i]$ and $E[i]$ denote the base and experimental marks obtained by the ith student, where $0 \leq i < N$. We use the terms "Base" and "Experiment" dataset with the belief that the metrics and techniques used for comparing the grading performances of an LLM grader vs a Human grader can be used to confirm the degree of affirmation between any two graders regardless of their nature, where the Experiment Grader is compared to a Base Grader in order to assess agreement. We note that for evaluation on a method level (as seen in PRE), the experiment dataset E is two dimensional, where for each student i, $E[i]$ contains M data points, where M is the number of methods being evaluated, and $E[i][m]$ denotes the marks obtained by student i for the method m. To compute metrics for an Experiment Dataset E that lies on a different scale versus B, we scale E linearly as

$$E_s[i] = E[i] \times \frac{R}{R_E}, \qquad 0 \leq i < N$$

```
"Selected rubric" Solution 1
"Feedback with marks"
1. Initialize a map or dictionary or array to store the character frequencies. Initially, the frequency
   for each character should be zero. [1 mark]
    -The student initializes an array `charCount` to store character frequencies, which is correct.[1 mark]
2. For each character in the first string, the corresponding frequency is incremented by 1.[1 mark]
    -The student correctly increments the frequency for each character in the first string `s1` using
     `charCount[s1.charAt(i)]+`. [1 mark]
3. For each character in the second string, decrement its corresponding frequency by 1. [1 mark]
    -The student correctly decrements the frequency for each character in the second string 's2' using
     'charCount[s2.charAt(i)]--'. [1 mark]
4. Iterate through the entire map or dictionary or array. If any frequency is non-zero, then return false.
   Else return true. [1 mark]
    -The student iterates through the `charCount array and checks if any frequency is non-zero, returning
     false if so, and true otherwise. [1 mark]
"Marks Scored out of total marks"
4 out of 4 marks
```

Figure 8: Sample Grade and Feedback for DSA

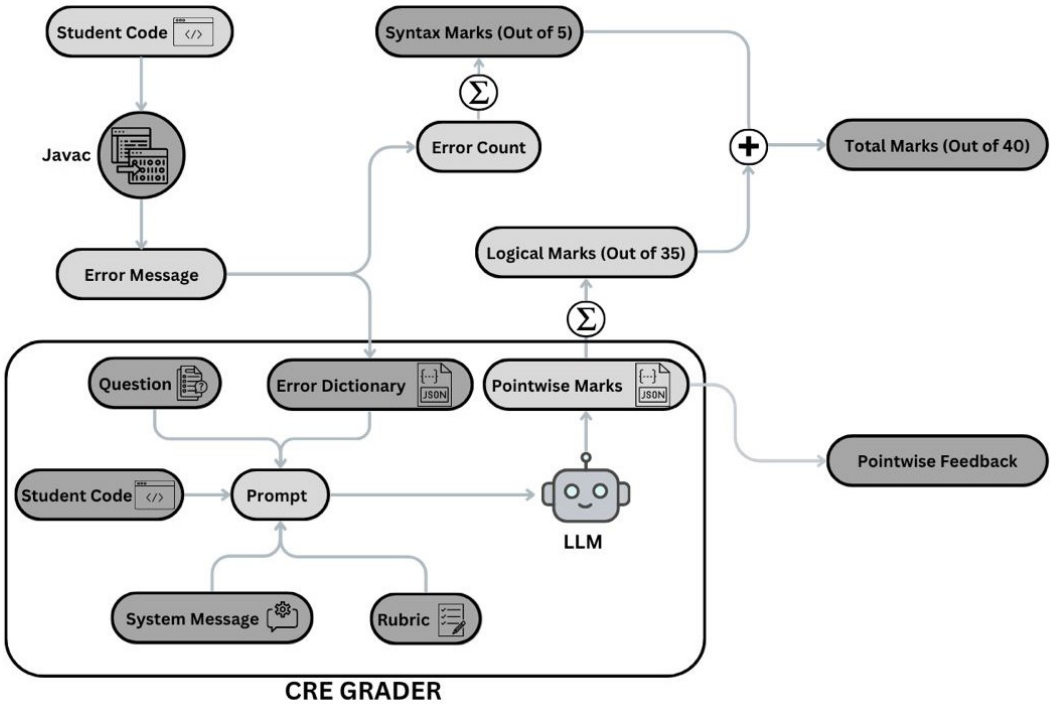

Figure 9: Complete Rubric Evaluation (CRE)

where R_E (e.g. 4 for the question-agnostic rubric) is the scale of the rubric being used for the evaluation of experiment marks, R is the scale of the question-specific rubric (also the scale of ground-truth/base values, e.g. 35 for the OOP dataset), N is the number of data points in the dataset, and E_s is the scaled experimental dataset.

For techniques evaluated method-by-method on the OOP dataset using question-agnostic rubrics, we scaled E on the method level as

$$E_s[i] = \sum_{m=1}^{M} \left(E[i][m] \times \frac{R[m]}{R_E}\right), \qquad 0 \leq i < N$$

where m is the method being evaluated, M is the number of methods present in the problem statement, and $R[m]$ is the maximum

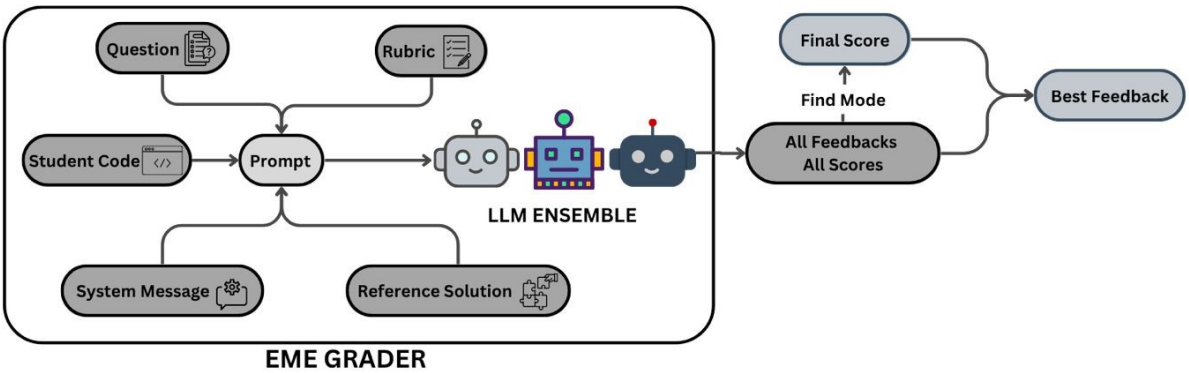

Figure 10: Ensembling Method Evaluation (EME)

number of marks that can be awarded for method m in the question-specific rubric. The total marks for a student are therefore the sum of marks awarded for each method, scaled up to match the base rubric.

The scaled experiment dataset E_s therefore eliminates the dimensionality issue presented by evaluating the experiment dataset E on a method level, by reducing it to a one dimensional dataset.

5.2.2 Binning. Before we obtain Cohen–Kappa scores on continuous datasets, we first classify the data into 5 bins. The bins are decided based on the ranks of the data points in the sorted datasets. For instance, the dataset $[3, 2, 4, 3, 1, 9, 6, 7]$ when classified into 3 bins is graded as $[0, 0, 1, 1, 0, 2, 1, 2]$, where 0, 1 and 2 are the three possible grades, assuming uniform grading. Concretely, each scaled score list is sorted and partitioned at the 20th, 40th, 60th and 80th percentiles, producing five ordinal intervals of roughly equal size. Marks falling below a cut-off receive the lower bin label (0–4). We adopt five bins because (i) it parallels the familiar A–E grading scale used in educational studies, (ii) it maintains adequate cell counts for stable κ estimates with $N \approx 100$, and (iii) sensitivity checks with 4 and 6 bins shifted κ by no more than 0.02. We acknowledge that any discretisation can distort near-threshold scores, yet the empirical drift observed here is minor and does not affect qualitative conclusions.

5.3 Evaluation Metrics

We assess each rubric–LLM pair with seven complementary statistics that together expose ordering, bias and exact-score agreement. Although several of the following statistics are mathematically related, no single number captures *all* facets of grading quality. These seven metrics can be categorized into three broad dimensions:

(1) Relative ordering of students (correlations) Rank correlations: **Pearson Correlation Coefficient** (PCC, r) [4] captures linear alignment and is most sensitive to large outliers. **Spearman Rank Correlation Coefficient** (r_s) [4] tests whether the overall ranking is preserved regardless of spacing, while **Kendall-Tau Correlation Coefficient** (τ_b) [4] provides a tie-aware alternative that remains stable on small samples. High cross-correlation among some measures (e.g. Pearson vs. Spearman) signals convergent validity, while the divergences highlight specific failure modes such as monotonic but non-linear trends or category-level disagreement after binning—that would be invisible in a single score.

(2) Agreement coefficients: **Intraclass Correlation Coefficient (ICC)** [2] comes in three flavours: ICC1 (one-way random), ICC2 (two-way random) and ICC3 (two-way mixed). These differ in how they treat the raters (randomly drawn versus fixed). All express absolute agreement in the original mark scale. **Cohen-Kappa (κ_B)** [3] treats scores as categories (bins), correcting for chance agreement; useful when instructors ultimately issue letter grades.

(3) Absolute bias or strictness: **Leniency (Mean Normalized Error, l_n))** between two datasets is defined as

$$l_n = \frac{\sum_{i=0}^{N-1}(\hat{E}[i] - \hat{B}[i])}{N},$$

where $\hat{E}[i]$ and $\hat{B}[i]$ are the experimental and base scores normalised to 1. Leniency is a measure of how strict the evaluation system is for a rubric. Leniency is relative, and requires base data against which the metric is computed. A perfectly lenient grader would assign R marks where every base data point is 0, achieving a leniency score of 1. A perfectly strict grader would assign 0 marks where every base data point is R, achieving a leniency score of −1. If we consider a target average deviation of 10%, the leniency to aim for would be within ±0.1.

Why Leniency matters? Correlation metrics (Pearson, Spearman, Kendall) reward graders that preserve the student ranking even when every mark is shifted by a constant amount. Leniency exposes that uniform bias: a positive l_n signals generosity, a negative l_n strictness. Considering both together separates disagreements caused by level shifts (bias) from those caused by rank dispersion. In practice we sometimes observe $r > 0.9$ while $|l_n| \pm 0.15$, indicating that a simple post-hoc rescaling (not rubric redesign) can reconcile the two graders.

6 Results

6.1 Techniques Used

- **CodeBERTScore** [46]: CodeBERTScore is an Automatic Evaluation Metric for Code evaluation, based on BERTScore. We evaluate CodeBERTScore on the OOP dataset and present correlation metrics. Leniency for CodeBERTScore is not evaluated, since the data points lie on a different scale versus the ground truth score.
- **CodeJudge** [41]: CodeJudge is an LLM-based evaluation technique . We implement the binary evaluation module in CodeJudge and evaluate binary scoring for each method. Once the score is obtained, we evaluate complete or zero marks for that method based on the marks in our rubric. CodeJudge achieves comparable results to those presented in the original paper.
- **Ensembling Method Evaluation (QA/QS)**: We use LLM ensembling to achieve results with a degree of consensus. Similar to CodeJudge, we evaluate each method in the original rubric, but instead of binary marking, we grade based on a 5 point rubric, awarding scores from 0-4. These marks are then scaled to 35 before evaluations.
- **CRE/PRE**: Complete and Pointwise Rubric Evaluation are question-specific rubric evaluation techniques discussed earlier in the paper. CRE and PRE are both evaluated on the original 35-mark rubric. For evaluation metrics on OOP, we consider only logical marks, as human grading of the OOP dataset followed a similar logical correctness-based approach.
- **Five Point Marking (FPM)** [34]: Five Point Marking is a question-agnostic LLM-based rubric marking technique. The student code is evaluated out of 100 split into the following categories:
  - Program Format (10 Marks)
  - Time Complexity (15 Marks)
  - Space Complexity (15 Marks)
  - Correctness General (30 Marks)
  - Correctness Edge Cases (30 Marks)

The scores are scaled down to 35 for comparison with ground truth values.

6.2 Findings

Tables 2 and 3 provide an overview of the performance of all implemented techniques on the OOP and DSA datasets.

6.2.1 *Overall Observations (Table 2 and Table 3).*

- **Presence of LLM Grader**: We observe that LLM-based techniques outperform CodeBERTScore significantly and impressively. CodeBERTScore obtains weak scores across all evaluation metrics. LLMs are vastly more versatile when it comes to understanding context, as well as the varying nature of student approaches to one particular problem.
- **Presence of Rubric:** We observe that both question-agnostic and question-specific rubric techniques outperform no-rubric technique (CodeJudge). While CodeJudge achieves respectable correlation and κ_B scores, providing a rubric gives the LLM grader an anchor around which to evaluate or grade the code, thereby increasing performance significantly when a rubric is provided

6.2.2 *Specific Observations from Results on DSA Dataset (Table 3).*

- **Question-specific beats question-agnostic rubric-based approaches:** On the algorithmically diverse and hard DSA dataset, moving from a question-agnostic rubric *(EME-QA)* to a question-specific rubric *(EME-QS)* lifts ICC3 from $0.560 \rightarrow 0.819$ and boosts Pearson r by +0.26 points ($0.562 \rightarrow 0.825$).
- **Type of Question-Agnostic rubric:** Using EME with a QA rubric provides significantly better correlation scores versus the FPM technique. The FPM technique attempts to grade the solution over 5 predefined marking points. An LLM when provided with only a problem statement and a solution code snippet is unable to grade accurately based on these 5 marking points, thereby yielding lower scores. In contrast, it performs better when asked to gauge the degree of correctness of the entire code snippet directly.

6.2.3 *Specific Observations from OOP Results (Table 2).*

- **Both question-specific and question-agnostic rubrics achieve comparable results for EME:** We see comparable performance for question-specific (EMA(QS)) and question-agnostic (EMA(QA)) rubric-based approaches across all metrics. As the OOP dataset contains homogeneous implementation-oriented questions, the results indicate that such questions are graded reliably even with question-agnostic rubrics.
- **Strictness depends on prompt granularity:** As shown in Table 2, *PRE* (feeding one rubric point at a time) slashes average scores by 11.5/35 marks (leniency = -0.329), whereas *CRE* (feeding the whole rubric) hovers near human leniency (0.081).
- **Why PRE is harsher than CRE?** Single-criterion prompts force the model to assign zero unless the exact logic is present, whereas whole-rubric prompts let it award partial credit, mirroring human evaluator behaviour. PRE seems to be more suitable for use cases requiring stringent evaluation and strict rubric adherence.

6.2.4 *EME Performance.*

We conducted an additional set of experiments to understand the impact of model parameter size and ensemble size for EME.

- **Model Parameter Size**: The performance of EME improves as the parameter size of the underlying language model increases. Larger models tend to demonstrate enhanced reasoning and evaluation capabilities, resulting in more accurate scoring and feedback generation. Conversely, models with relatively lower parameter counts exhibit greater inconsistencies, leading to lower correlation.
- **Ensemble Size**: The effectiveness of the method varies with ensemble size. Initially, a significant increase in correlation coefficients is observed as ensemble size increases. This trend continues until the ensemble reaches an optimal range—typically around three to four models when employing high-parameter LLMs, such as GPT-4 and Claude 3. Beyond this threshold, performance gains tend to plateau, suggesting diminishing returns with further increase in ensemble size.

Method	r	r_s	τ_b	l_n	ICC1	ICC2	ICC3	κ_B
No LLM (Similarity Based)								
CodeBERTScore	0.354	0.482	0.343	-	-	-	-	0.241
No Rubric								
CodeJudge	0.717	0.745	0.597	-0.233	0.479	0.541	0.712	0.433
Question-Agnostic (QA) Rubric								
EME (QA)	0.904	**0.909**	**0.775**	-0.071	**0.881**	**0.882**	0.904	0.512
FPM	0.844	0.834	0.670	-0.121	0.778	0.784	0.834	0.346
Question-Specific (QS) Rubric								
EME (QS)	0.900	0.902	0.769	-0.067	0.878	0.880	0.900	0.545
CRE (OpenAI)	**0.912**	0.906	0.773	0.082	0.880	0.882	**0.910**	**0.598**
CRE (Claude)	0.840	0.841	0.696	**-0.008**	0.841	0.841	0.840	0.572
PRE	0.742	0.795	0.637	-0.329	0.201	0.378	0.679	0.302

Table 2: Results obtained by various techniques on the OOP dataset

Method	r	r_s	τ_b	l_n	ICC1	ICC2	ICC3	κ_B
No LLM								
CodeBERTScore	0.126	0.058	0.039	-	-	-	-	0.010
No Rubric								
CodeJudge	0.423	0.427	0.389	-0.315	0.176	0.272	0.353	0.406
Question-Agnostic (QA) Rubric								
EME (QA)	0.562	0.510	0.445	-0.098	0.509	0.525	0.560	0.156
FPM	0.470	0.381	0.320	-0.054	0.380	0.388	0.398	0.072
Question-Specific (QS) Rubric								
EME (QS)	**0.825**	**0.763**	**0.675**	**0.0049**	**0.821**	**0.820**	**0.819**	**0.646**

Table 3: Results obtained by various techniques on the DSA dataset

7 Discussion

7.1 Implications for Instructors and Teaching Assistants

For instructors and teaching assistants, the three graders (CRE, PRE and EME) can be combined in a staged workflow. A lightweight CRE pass may quickly triage submissions, flagging clearly strong or weak attempts, whereas stricter PRE scoring (or a brief human check) can be reserved for borderline cases. In settings where false positives carry a high cost, such as summative examinations, PRE (strict) may offer additional reassurance, trading a slight reduction in overall agreement for a zero-tolerance stance on partial logic. Routine coursework and large MOOC cohorts, by contrast, often prioritise speed and cost. For such scenarios, CRE seems to be a reasonable cost-effective choice. Classes that attract highly diverse algorithmic solutions (for example, DSA) could find value in EME (QS), which attained the highest ICC3 in our DSA benchmark, albeit at the price of a three-model ensemble. We also note that for larger, easier and more direct problems, as seen in the OOP dataset, a simple question-agnostic rubric with method-wise evaluation produces near-human-like grading and is on par with evaluation using question-specific rubrics. It does, however, rely on multiple LLM calls (once for each method) rather than a single LLM call with a large rubric. Developing a question-specific rubric may therefore save token costs during evaluation.

The JSON traces emitted by all three techniques may also feed into analytics dashboards. Aggregating the most frequently missed rubric criteria can highlight common misconceptions, enabling teaching teams to design targeted recitations, micro-lectures, or discussion-board posts that address exactly those weak spots.

A minimal integration path could involve: (i) translating an existing rubric into the key–value format accepted by the graders, (ii) calibrating Leniency or ensemble size on roughly ten pilot submissions until the mean-normalised error falls within ±0.1, (iii) inserting the grader call into the continuous-integration script that already compiles and tests student code, and (iv) offering reviewers an interface that surfaces rubric items with confidence below 0.8 for optional human override.

7.2 Guidance for Students

Rubric-aligned feedback provides criterion-level transparency: each comment is anchored to a specific step (e.g., Step 6 – create a `Player` object), helping learners understand *why* marks were lost instead of receiving a generic wrong output notice. Encouraging students to summarise the feedback in a simple worksheet (criterion missed, probable cause, planned fix) may promote systematic debugging. Learners might further consolidate gains by submitting a short reflection after resubmission detailing which rubric elements they have mastered, which remain challenging, and what strategies (additional unit tests, peer review, etc.) they will employ next time. Such

reflective practice aligns with accepted principles of self-regulated learning.

8 Limitations and Future Work

Although this study provides valuable insights into rubric-based code evaluation, certain limitations must be acknowledged which also advocate avenues for future work. Firstly, we didn't explore variability among various LLMs and mainly worked with GPT-4o. Different training methodologies may influence effectiveness and accuracy of code evaluation and a future research could compare multiple models. We also didn't explore the differences between open-sources and closed-source LLMs. Our analysis is limited to programming questions in Java and do not test other widely-used languages like Python, C++. We focussed on two intermediary courses which had single file code assignments. A future research could delve into advanced courses which require multiple files handling. The rubrics can be of various granularities ranging from low to medium to fine. A detailed study on their effectiveness and comparison is another future avenue for research.

9 Conclusion

In this paper, we explored how effectively can an LLM-based grader evaluate student code using a question-specific rubric compared to a question-agnostic rubric. To facilitate our evaluation, we introduced two novel datasets focused on DSA and OOP, encompassing solutions of varying correctness levels. Using these datasets, we assessed our proposed techniques against existing approaches using exisitng and a new evaluation metric, Leniency. Our findings highlight the limitations of existing evaluation techniques and demonstrate the potential benefits of question-specific rubrics in enhancing logical assessment of code. This work not only provides new insights into LLM-based code evaluation but also lays the groundwork for future research into refining evaluation methods and expanding dataset availability to improve automated code assessment.

Acknowledgments

This research was carried out in part with support from the New Faculty Seed Grant, Birla Institute of Technology and Science (BITS), Pilani (Grant Ref. N4/24/1004).

The authors also wish to acknowledge the use of ChatGPT/-Claude in the writing of this paper. This tool was used to generate ideas regarding the presentation of tables and figures in the paper, and to improve the written grammar. The paper remains an accurate representation of the authors' underlying work and novel intellectual contributions.

References

[1] [n. d.]. Claude 3.7 Sonnet and Claude Code. https://www.anthropic.com/news/claude-3-7-sonnet

[2] 2024. *Pingouin*. https://github.com/raphaelvallat/pingouin/releases/tag/0.5.5 Release 0.5.5 from September 2024.

[3] 2024. *scikit-learn*. https://github.com/scikit-learn/scikit-learn/releases/tag/1.6.1 Release 1.6.1.

[4] 2025. *SciPy*. https://github.com/scipy/scipy/releases/tag/v1.15.2 Release 1.15.2 from Feb 2025.

[5] Umar Alkafaween, Ibrahim Albluwi, and Paul Denny. 2024. Automating Autograding: Large Language Models as Test Suite Generators for Introductory Programming. *Journal of Computer Assisted Learning* 41, 1 (Dec. 2024). https://doi.org/10.1111/jcal.13100

[6] Anishka, Atharva Mehta, Nipun Gupta, Aarav Balachandran, Dhruv Kumar, and Pankaj Jalote. 2024. Can ChatGPT Play the Role of a Teaching Assistant in an Introductory Programming Course? arXiv:2312.07343 [cs.HC] https://arxiv.org/abs/2312.07343

[7] Imen Azaiz, Natalie Kiesler, and Sven Strickroth. 2024. Feedback-Generation for Programming Exercises With GPT-4. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024)*. ACM, 31–37. https://doi.org/10.1145/3649217.3653594

[8] Faieza Chowdhury. 2018. Application of Rubrics in the Classroom: A Vital Tool for Improvement in Assessment, Feedback and Learning. *International Education Studies* (2018). https://api.semanticscholar.org/CorpusID:59295385

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL] https://arxiv.org/abs/1810.04805

[10] Zhiyuan Fan, Weinong Wang, Xing Wu, and Debing Zhang. 2025. SedarEval: Automated Evaluation using Self-Adaptive Rubrics. arXiv:2501.15595 [cs.CV] https://arxiv.org/abs/2501.15595

[11] Qiang Hao and Michail Tsikerdekis. 2019. How Automated Feedback is Delivered Matters: Formative Feedback and Knowledge Transfer. In *2019 IEEE Frontiers in Education Conference (FIE)*. 1–6. https://doi.org/10.1109/FIE43999.2019.9028686

[12] Arto Hellas, Juho Leinonen, and Leo Leppänen. 2024. Experiences from Integrating Large Language Model Chatbots into the Classroom. arXiv:2406.04817 [cs.CY] https://arxiv.org/abs/2406.04817

[13] Arto Hellas, Juho Leinonen, Sami Sarsa, Charles Koutcheme, Lilja Kujanpää, and Juha Sorva. 2023. Exploring the Responses of Large Language Models to Beginner Programmers' Help Requests. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1 (ICER 2023)*. ACM, 93–105. https://doi.org/10.1145/3568813.3600139

[14] Yann Hicke, Anmol Agarwal, Qianou Ma, and Paul Denny. 2023. AI-TA: Towards an Intelligent Question-Answer Teaching Assistant using Open-Source LLMs. arXiv:2311.02775 [cs.LG] https://arxiv.org/abs/2311.02775

[15] Petri Ihantola, Tuukka Ahoniemi, Ville Karavirta, and Otto Seppälä. 2010. Review of recent systems for automatic assessment of programming assignments. In *Proceedings of the 10th Koli Calling International Conference on Computing Education Research* (Koli, Finland) *(Koli Calling '10)*. Association for Computing Machinery, New York, NY, USA, 86–93. https://doi.org/10.1145/1930464.1930480

[16] Sven Jacobs and Steffen Jaschke. 2024. Evaluating the Application of Large Language Models to Generate Feedback in Programming Education. In *2024 IEEE Global Engineering Education Conference (EDUCON)*. IEEE, 1–5. https://doi.org/10.1109/educon60312.2024.10578838

[17] Ishika Joshi, Ritvik Budhiraja, Harshal Dev, Jahnvi Kadia, Mohammad Osama Ataullah, Sayan Mitra, Harshal D. Akolekar, and Dhruv Kumar. 2024. ChatGPT in the Classroom: An Analysis of Its Strengths and Weaknesses for Solving Undergraduate Computer Science Questions. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (Portland, OR, USA) *(SIGCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 625–631. https://doi.org/10.1145/3626252.3630803

[18] Ishika Joshi, Ritvik Budhiraja, Pranav Deepak Tanna, Lovenya Jain, Mihika Deshpande, Arjun Srivastava, Srinivas Rallapalli, Harshal D Akolekar, Jagat Sesh Challa, and Dhruv Kumar. 2023. "With Great Power Comes Great Responsibility!": Student and Instructor Perspectives on the influence of LLMs on Undergraduate Engineering Education. arXiv:2309.10694 [cs.HC] https://arxiv.org/abs/2309.10694

[19] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2016. Towards a Systematic Review of Automated Feedback Generation for Programming Exercises. In *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education* (Arequipa, Peru) *(ITiCSE '16)*. Association for Computing Machinery, New York, NY, USA, 41–46. https://doi.org/10.1145/2899415.2899422

[20] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A Systematic Literature Review of Automated Feedback Generation for Programming Exercises. *ACM Trans. Comput. Educ.* 19, 1, Article 3 (Sept. 2018), 43 pages. https://doi.org/10.1145/3231711

[21] Natalie Kiesler, Dominic Lohr, and Hieke Keuning. 2023. Exploring the Potential of Large Language Models to Generate Formative Programming Feedback. arXiv:2309.00029 [cs.AI] https://arxiv.org/abs/2309.00029

[22] Nachiket Kotalwar, Alkis Gotovos, and Adish Singla. 2025. Hints-In-Browser: Benchmarking Language Models for Programming Feedback Generation. arXiv:2406.05053 [cs.LG] https://arxiv.org/abs/2406.05053

[23] Charles Koutcheme, Nicola Dainese, Sami Sarsa, Arto Hellas, Juho Leinonen, Syed Ashraf, and Paul Denny. 2025. Evaluating Language Models for Generating and Judging Programming Feedback. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1* (Pittsburgh, PA, USA) *(SIGCSETS 2025)*. Association for Computing Machinery, New York, NY, USA, 624–630. https://doi.org/10.1145/3641554.3701791

[24] Mark Liffiton, Brad Sheese, Jaromir Savelka, and Paul Denny. 2023. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. arXiv:2308.06921 [cs.CY] https://arxiv.org/abs/2308.06921

[25] Xiao Liu, Shuai Wang, Pei Wang, and Dinghao Wu. 2019. Automatic Grading of Programming Assignments: An Approach Based on Formal Semantics. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. 126–137. https://doi.org/10.1109/ICSE-SEET.2019.00022

[26] Richard Lobb and Jenny Harlow. 2016. Coderunner: a tool for assessing computer programming skills. *ACM Inroads* 7, 1 (Feb. 2016), 47–51. https://doi.org/10.1145/2810041

[27] B. Mandernach, Swinton Hudson, and Shanna Wise. 2013. Where Has The Time Gone? Faculty Activities and Time Commitments in the Online Classroom. *Journal of Educators Online* 10 (07 2013). https://doi.org/10.9743/JEO.2013.2.2

[28] Marcus Messer, Neil C. C. Brown, Michael Kölling, and Miaojing Shi. 2024. Automated Grading and Feedback Tools for Programming Education: A Systematic Review. *ACM Transactions on Computing Education* 24, 1 (Feb. 2024), 1–43. https://doi.org/10.1145/3636515

[29] Dan-Anders Normann, Lise Vikan Sandvik, and Henning Fjørtoft. 2023. Reduced grading in assessment: A scoping review. *Teaching and Teacher Education* 135 (2023), 104336. https://doi.org/10.1016/j.tate.2023.104336

[30] OpenAI. 2023. GPT-4 Technical Report. https://arxiv.org/abs/2303.08774v2

[31] Christiano Otero Avila, Luciana Foss, Adriana Bordini, Maria Simone Debacco, and Simone André da Costa Cavalheiro. 2019. Evaluation Rubric for Computational Thinking Concepts. In *2019 IEEE 19th International Conference on Advanced Learning Technologies (ICALT)*, Vol. 2161-377X. 279–281. https://doi.org/10.1109/ICALT.2019.00089

[32] Maciej Pankiewicz and Ryan S. Baker. 2023. Large Language Models (GPT) for automating feedback on programming assignments. arXiv:2307.00150 [cs.HC] https://arxiv.org/abs/2307.00150

[33] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, Pierre Isabelle, Eugene Charniak, and Dekang Lin (Eds.). Association for Computational Linguistics, Philadelphia, Pennsylvania, USA, 311–318. https://doi.org/10.3115/1073083.1073135

[34] Tung Phung, Victor-Alexandru Pădurean, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generative AI for Programming Education: Benchmarking ChatGPT, GPT-4, and Human Tutors. arXiv:2306.17156 [cs.CY] https://arxiv.org/abs/2306.17156

[35] Vreda Pieterse. 2013. Automated Assessment of Programming Assignments. In *Proceedings of the 3rd Computer Science Education Research Conference on Computer Science Education Research* (Arnhem, Netherlands) *(CSERC '13)*. Open Universiteit, Heerlen, Heerlen, NLD, 45–56.

[36] James Prather, Raymond Pettit, Kayla Holcomb McMurry, Alani Peters, John Homer, Nevan Simone, and Maxine Cohen. 2017. On Novices' Interaction with Compiler Error Messages: A Human Factors Approach. In *Proceedings of the 2017 ACM Conference on International Computing Education Research* (Tacoma, Washington, USA) *(ICER '17)*. Association for Computing Machinery, New York, NY, USA, 74–82. https://doi.org/10.1145/3105726.3106169

[37] Nishat Raihan, Mohammed Latif Siddiq, Joanna C. S. Santos, and Marcos Zampieri. 2024. Large Language Models in Computer Science Education: A Systematic Literature Review. arXiv:2410.16349 [cs.LG] https://arxiv.org/abs/2410.16349

[38] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. arXiv:2009.10297 [cs.SE] https://arxiv.org/abs/2009.10297

[39] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research - Volume 1 (ICER 2022)*. ACM, 27–43. https://doi.org/10.1145/3501385.3543957

[40] Hyein Seo, Taewook Hwang, Jeesu Jung, Hyeonseok Kang, Hyuk Namgoong, Yohan Lee, and Sangkeun Jung. 2025. Large Language Models as Evaluators in Education: Verification of Feedback Consistency and Accuracy. *Applied Sciences* 15, 2 (2025). https://doi.org/10.3390/app15020671

[41] Weixi Tong and Tianyi Zhang. 2024. CodeJudge: Evaluating Code Generation with Large Language Models. arXiv:2410.02184 [cs.LG] https://arxiv.org/abs/2410.02184

[42] En-Qi Tseng, Pei-Cing Huang, Chan Hsu, Peng-Yi Wu, Chan-Tung Ku, and Yihuang Kang. 2024. CodEv: An Automated Grading Framework Leveraging Large Language Models for Consistent and Constructive Feedback. In *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 5442–5449. https://doi.org/10.1109/bigdata62323.2024.10825949

[43] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) *(CHI EA '22)*. Association for Computing Machinery, New York, NY, USA, Article 332, 7 pages. https://doi.org/10.1145/3491101.3519665

[44] Wenjing Xie, Juxin Niu, Chun Jason Xue, and Nan Guan. 2024. Grade Like a Human: Rethinking Automated Assessment with Large Language Models. arXiv:2405.19694 [cs.AI] https://arxiv.org/abs/2405.19694

[45] Mina Yousef, Kareem Mohamed, Walaa Medhat, Ensaf Hussein Mohamed, Ghada Khoriba, and Tamer Arafa. 2024. BeGrading: large language models for enhanced feedback in programming education. *Neural Computing and Applications* 37, 2 (Oct. 2024), 1027–1040. https://doi.org/10.1007/s00521-024-10449-y

[46] Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. 2023. CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code. arXiv:2302.05527 [cs.SE] https://arxiv.org/abs/2302.05527

[47] Terry Yue Zhuo. 2024. ICE-Score: Instructing Large Language Models to Evaluate Code. arXiv:2304.14317 [cs.AI] https://arxiv.org/abs/2304.14317

A Prompts for Techniques

A.1 CRE

```
You are an expert code evaluator, evaluating code submissions for a Java based Object Oriented Programming test at a university level.
You will be provided with the question and a rubric that describes the criteria for evaluation, with a marking scheme.
The question is a code sample that the examiner provides, containing a template wherein the student is required to write the code as well as comments and instructions from the examiner's end.
Following this you will be provided with the code submission, along with the response from the Java compiler that runs this code.
Note that the code may be formatted liberally, the specific positioning of the code within the methods are not important.
Code may be present either before or after the comments prepared by the instructor.
You are to evaluate the code based only on logical correctness. You are to ignore any syntax errors that the compiler may have thrown.
Any syntax errors that you encounter can be treated as correct syntax, and you are to infer the student's logical flow and intention from the code.
You are to return your response as a JSON dictionary containing a detailed, nested evaluation of the student's marks for each line in the rubric.
The JSON dictionary should also contain feedback for each point in the rubric.
For each line in the rubric, you are to provide the line as the key and a nested dictionary containing marks awarded and feedback.
The following is a sample return Format:
{
"1000": {
    "Method1": {
        "Point1": {
            "Marks": 3,
            "Feedback": {LLM obtained feedback}
        },
        "Point2":{...},
        ...
    },
    "Method2: {...},
    ...
}
}
DO NOT RETURN ANY ADDITIONAL TEXT ASIDE FROM THE JSON DICTIONARY.
Question: {}
Rubric: {}
Code Submission: {}
Compiler Response: {}
```

A.2 PRE

```
You are an expert code evaluator, evaluating code submissions for a Java based Object Oriented Programming test at a university level.
You will be provided with the question, the code snippet, and the point of evaluation for the code. You will also be given the compiler response for the code.
You will also be given the rubric point that the student is graded on. You are to evaluate based on that particular point only.
The question is a code sample that the examiner provides, containing a template wherein the student is required to write the code as well as comments and instructions from the examiner's end.
Following this you will be provided with the code submission, along with the response from the Java compiler that runs this code.
Note that the code may be formatted liberally, the specific positioning of the code within the methods are not important.
Code may be present either before or after the comments prepared by the instructor.
You are to evaluate the code based only on logical correctness. You are to ignore any syntax errors that the compiler may throw.
Any syntax errors that you encounter can be treated as correct syntax, and you are to infer the student's logical flow and intention from the code.
You are to return only a dictionary containing the your decision and your feedback, with the keys "DECISION" and "FEEDBACK"
For your decision, return YES if the student has correctly implemented the logic for the given rubric point, and NO if they have not.
Since there is no partial marking and we're only considering logical correctness, be liberal with the quality of the code and the marking.
The following is a sample return Format:
{
    "DECISION": "YES",
    "FEEDBACK": {LLM obtained Feedback},
}
DO NOT RETURN ANY ADDITIONAL TEXT ASIDE FROM THE DICTONARY.
Question: {}
Student Solution: {}
Point to be evaluated: {}
Compiler Response: {}
```

A.3 EME

```
## Approach Identification Prompt

You are analyzing a student's code submission for a DSA problem.
Based on the rubric, identify which approach the student is using.

Rubric:
```
{rubric_content}
```

Student Code:
```
{code}
```

IMPORTANT INSTRUCTIONS:
1. The rubric contains multiple solution approaches (e.g., "Solution 1", "Solution 2", "Solution 3").
2. Each approach has specific criteria and point allocations.
3. Carefully analyze the student's code to determine which approach they are using.
4. Look for key patterns, variable names, and algorithm structures that match one of the approaches in the rubric.
5. The approach name should be EXACTLY as it appears in the rubric (e.g., "Brute Force", "Dynamic Programming", "Kadane's Algorithm").

Respond ONLY with a JSON object in this exact format:
{
    "identified_approach": "Exact approach name from rubric (e.g., 'Solution 1 (Brute Force)', 'Solution 3 (Kadane's Algorithm)')",
    "confidence": 0.95, // A number between 0 and 1 indicating confidence in the identification
    "reasoning": "Brief explanation of why you identified this approach, citing specific code patterns that match the rubric criteria"
}

## System Message for Approach Identification
You are a code analyzer that ONLY responds with valid JSON. No other text or explanation. You must identify the exact approach from the rubric.

## Code Evaluation Prompt

You are evaluating a student's code submission for a DSA problem.
Provide your evaluation in VALID JSON format only.

Problem:
```
{question}
```

Rubric:
```
{rubric}
```

Reference Solution:
```
{solution}
```

Student Code:
```
{code}
```

The student appears to be using the "{identified_approach}" approach.

IMPORTANT INSTRUCTIONS:
1. Evaluate the submission according to the EXACT criteria in the rubric for this approach.
2. For each criterion in the rubric for this approach, assign appropriate points.
3. The criterion descriptions should match EXACTLY what's in the rubric.
4. The max_score for each criterion should match the points specified in the rubric.
5. Your feedback should directly address how well the student's code meets each specific criterion.
6. Do not create new criteria that aren't in the rubric.

Respond ONLY with a JSON object in this exact format:
{
    "criteria_scores": [
        {"criterion": "exact criterion from rubric", "score": awarded_points, "max_score": points_specified_in_rubric, "feedback": "specific feedback for this criterion"}
    ],
    "total_score": total_awarded_points,
    "max_possible_score": total_maximum_points,
    "overall_feedback": "overall feedback here",
```

```
    "approach_correctness": 0.95, // How confident you are that the approach identification is correct (0-1)
    "code_correctness": 0.9, // How likely the code is to work correctly (0-1)
    "efficiency_rating": 0.8, // How efficient the solution is relative to optimal (0-1)
    "readability_rating": 0.7 // How readable and well-structured the code is (0-1)
}

## System Message for Code Evaluation
You are a code evaluator that ONLY responds with valid JSON. No other text or explanation. You must follow the rubric exactly when evaluating code.
```